

Study notes: Inverted Indexing, Sparse Retrieval and Vocabulary Mismatch

Everything behind the five parts of the assignment and every script in `cs6101/`

September 2026

Contents

1	The whole pipeline in one picture	2
2	Setup and how to run	2
3	Shared code: <code>datasets.py</code> and <code>common.py</code>	2
4	Part 1: building the index (<code>index.py</code>, <code>index_stats.py</code>)	3
4.1	Concept	3
4.2	Code	3
5	Part 2: BM25, tuning, TF-IDF (<code>phase2.py</code>)	4
5.1	Concept	4
5.2	Code	4
6	Part 3: seeing vocabulary mismatch (<code>phase3.py</code>)	5
6.1	Concept	5
6.2	Code	5
7	Part 4a: Rocchio and RM3 (<code>rocchio.py</code>, <code>phase4a.py</code>)	6
7.1	Concept	6
7.2	Code	6
8	Part 4b: HyDE (<code>hyde_generate.py</code>, <code>phase4b.py</code>)	7
8.1	Concept	7
8.2	Code	7
9	Part 5: SPLADE (<code>phase5.py</code>)	8
9.1	Concept	8
9.2	Code	8
10	Reading the results	8

1 The whole pipeline in one picture

Every part of the assignment is one stage of the same pipeline:

1. **Data:** BEIR corpora (documents), test queries, and *qrels* (which documents are relevant to which query).
2. **Index** (Part 1): turn the documents into a Lucene inverted index so a query can be answered in milliseconds.
3. **Retrieve** (Part 2): score documents against a query with BM25 or TF-IDF and evaluate the ranking with nDCG, Recall, MRR, MAP.
4. **Diagnose** (Part 3): look at the queries BM25 gets wrong and measure how much they share words with their gold document.
5. **Expand the query** (Part 4): add words to the query, taken either from the top retrieved documents (Rocchio, RM3) or from text an LLM writes (HyDE).
6. **Learn the expansion** (Part 5): SPLADE, a neural model that produces the sparse word-weight vector itself.

All scripts live in one folder, share `common.py` and `datasets.py`, write JSON to `results/`, and `make_tables.py` turns the JSON into the LaTeX tables used by both reports. `run_all.sh` runs the scripts in order, one at a time (the laptop has 7.6 GB of RAM for WSL and no GPU; two heavy jobs at once thrash or get killed by the OOM killer).

2 Setup and how to run

- WSL2 Ubuntu, Python 3.12, Java 21, `pyserini==2.3.0` (bundles Anserini and Lucene 10), `ir_datasets`, `ir_measures`, PyTorch (CPU), Transformers, matplotlib, scipy.
- `python download.py` once: fetches the SPLADE checkpoint, the Qwen model, and the three prebuilt SPLADE indexes.
- `bash run_all.sh`: `index.py` → `index_stats.py` → `phase2.py` → `phase3.py` → `phase4a.py` → `hyde_generate.py` → `phase4b.py` → `phase5.py` → `make_tables.py`. Log in `logs/run_all.log`.
- Any script also takes dataset names: `python phase2.py scifact`.
- Compile: `pdflatex report.tex` (twice, for references) inside `report/`; same for `learning.tex`.

Two pitfalls met on this machine: a NUL byte written into the 1.7GB HotpotQA JSONL on the OneDrive mount made Anserini stop silently at 2.9M of 5.2M documents (fix: delete and regenerate the file; check `Total N documents indexed` in the log), and running the LLM, the SPLADE encoder and an index build together made the index build four times slower and triggered the OOM killer. Timings in the report were therefore measured with one job at a time.

3 Shared code: `datasets.py` and `common.py`

`datasets.py` maps short names to `ir_datasets` ids (`beir/scifact/test`, ...) and exposes `docs_iter()`, `queries_iter()`, `qrels_iter()`. A document has `doc_id`, `title`, `text`; a query has `query_id`, `text`; a qrel has `query_id`, `doc_id`, `relevance`.

`common.py`:

```
METRICS = [nDCG @ 10, R @ 100, RR @ 10, AP] # ir_measures objects
SAMPLE = 500 # queries for Parts 4-5 on FEVER / HotpotQA
```

```

def load(name, sample=None): # queries + qrels, optionally a fixed random sample
    ...
    random.Random(0).shuffle(queries) # seed 0 => same sample in every script
    queries = sorted(queries[:sample], key=lambda q: q.query_id)
def evaluate(run, qrels): # run = {qid: {docid: score}}
    return {str(m): round(float(v), 4) for m, v in calc_aggregate(METRICS, qrels, run).
            items()}
def search(searcher, queries, k=100):
    return hits_to_run(searcher.batch_search([q.text ...], [q.query_id ...], k=k, threads
    =8))

```

Why a sample? Parts 4 and 5 do work *per query* (a second search with a long weighted query, LLM generation, SPLADE encoding). On 5M-document corpora that is minutes per hundred queries on a CPU, so a fixed 500-query subset keeps the experiments feasible; the BM25 baseline is always recomputed on the same subset so comparisons stay fair. `save(part, name, data)` merges one dataset's result into `results/<part>.json`.

4 Part 1: building the index (`index.py`, `index_stats.py`)

4.1 Concept

An *inverted index* maps every term to a *postings list*: the documents containing it and the term frequency (tf) in each. Lucene also stores per document the length (for normalisation) and, only if asked (`-storeDocvectors`), the full term vector of each document, which is what Rocchio needs later. Before indexing, text goes through an *analyzer*: tokenise, lowercase, remove stop words, Porter-stem (*studies*, *studying* → *studi*). Queries go through the same analyzer, otherwise nothing would match.

4.2 Code

```

for doc in datasets.get_corpus(datasets.load_dataset(name)):
    f.write(json.dumps({"id": doc.doc_id, "contents": doc.title + "\n" + doc.text}) + "\n"
    )
subprocess.run(["python", "-m", "pyserini.index.lucene", "-collection", "JsonCollection",
    "-input", collection_dir, "-index", index_dir, "-storeDocvectors"], check=
    True)

```

Pyserini's `JsonCollection` wants one JSON object per line with `id` and `contents`. The indexer is a Java program, so it is launched as a subprocess and timed around the call; the directory size afterwards is the index size. `index_stats.py` then opens the index with `LuceneIndexReader`:

```

reader.stats() # documents, unique_terms, total_terms
reader.get_document_vector(docid) # {"cell": 5, "stem": 5, ...} analysed terms -> tf
sum(vec.values()) # document length in indexed tokens
reader.get_term_counts("cell", analyzer=None)[0] # df: documents containing the term

```

`analyzer=None` matters: the term from a document vector is already stemmed; running the analyzer again could stem it a second time and look up the wrong string.

Dataset	Documents	Test queries	Unique terms	Build time	Index size
SciFact	–	–	–	0.0 min	0.0 MB
FEVER	–	–	–	0.0 min	0.0 MB
HotpotQA	–	–	–	0.0 min	0.0 MB

Table 1: Part 1: Lucene index per dataset (Pyserini `JsonCollection`, `-storeDocvectors`, 4 indexing threads).

Dataset	Gold doc id	Length	Distinct terms	Top-tf terms (tf, corpus df)
---------	-------------	--------	----------------	------------------------------

Table 2: Part 1: IndexReader statistics for the first gold document of each dataset (analysed, stemmed terms).

5 Part 2: BM25, tuning, TF-IDF (phase2.py)

5.1 Concept

BM25 scores a document d for query q as

$$\sum_{t \in q} \text{idf}(t) \frac{\text{tf}(t, d) (k_1 + 1)}{\text{tf}(t, d) + k_1 (1 - b + b |d| / \text{avgdl})}, \quad \text{idf}(t) = \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}.$$

k_1 controls how quickly repeated occurrences stop adding score (saturation); $b \in [0, 1]$ controls how strongly long documents are penalised ($b=0$: no length normalisation, $b=1$: full). Pyserini’s default is $k_1=0.9$, $b=0.4$. Classic TF-IDF (Lucene’s `ClassicSimilarity`) uses $\sqrt{\text{tf}} \cdot \text{idf}^2$ with a cosine-style length norm: no saturation and a weaker, fixed length normalisation, which is why it is usually worse than BM25.

Metrics on a ranked list with relevance labels: **nDCG@10** = DCG of the top-10 ($\sum \text{rel}_i / \log_2(i+1)$) divided by the DCG of the ideal ordering; **Recall@100** = fraction of relevant documents found in the top-100; **MRR@10** = $1/\text{rank}$ of the first relevant document if it is within 10, else 0; **MAP** = mean over queries of the average of precision at each relevant document’s rank. All averaged over queries by `ir_measures`.

5.2 Code

```
searcher = LuceneSearcher(f"indexes/{name}")
searcher.set_bm25(0.9, 0.4)
for k1 in [0.6, 0.9, 1.2, 1.5, 2.0]:
    for b in [0.3, 0.4, 0.5, 0.75, 1.0]:
        searcher.set_bm25(k1, b)
        grid[(k1, b)] = common.evaluate(common.search(searcher, queries), qrels)
k1, b = max(grid, key=lambda p: grid[p]["nDCG@10"])
searcher.object.searcher.setSimilarity(ClassicSimilarity()) # Lucene IndexSearcher via
pyjnius
```

The similarity is a property of the searcher, not of the index, so TF-IDF needs no re-indexing.

Dataset	Model	nDCG@10	R@100	MRR@10	MAP
---------	-------	---------	-------	--------	-----

Table 3: Part 2: sparse baselines on the full BEIR test query sets.

Dataset	Best nDCG@10 (k_1, b)	Worst nDCG@10 (k_1, b)	Default	BM25 latency (ms/query)
---------	---------------------------	----------------------------	---------	-------------------------

Table 4: Part 2: grid search over $k_1 \in \{0.6, 0.9, 1.2, 1.5, 2.0\}$, $b \in \{0.3, 0.4, 0.5, 0.75, 1.0\}$ on all test queries (nDCG@10); latency = batch BM25 search with 8 threads.

6 Part 3: seeing vocabulary mismatch (phase3.py)

6.1 Concept

Vocabulary mismatch: the query and the relevant document say the same thing with different words (synonyms: *tumour/neoplasm*; paraphrase: *reduces the risk of/is protective against*; abbreviation: *MRI/magnetic resonance imaging*). A lexical model such as BM25 can only match identical (stemmed) strings. To measure it we use the *Jaccard coefficient* $J = |Q \cap D| / |Q \cup D|$ between the set of analysed query tokens and the set of terms in the gold document’s indexed vector. Because both sides are analysed the same way, J measures exactly what BM25 could match.

How strongly does low J predict a miss? We compare the J distribution of pairs where the gold document is in the top-10 (success) with pairs where it is not (failure), and report the *AUC*: the probability that a random success pair has a higher J than a random failure pair (0.5 = no signal, 1.0 = perfect). It equals the Mann–Whitney U statistic divided by $n_{\text{succ}} n_{\text{fail}}$, which is what `scipy.stats.mannwhitneyu` gives.

6.2 Code

```
ranked = sorted(run[q.query_id], key=run[q.query_id].get, reverse=True)
rank = ranked.index(docid) + 1 if docid in ranked else None
success = rank is not None and rank <= 10
jaccard = len(q_tok & d_tok) / len(q_tok | d_tok)
auc = mannwhitneyu(succ, fail).statistic / (len(succ) * len(fail))
```

Failure categories are a simple heuristic: no shared term $\rightarrow$ *no lexical overlap*; an all-caps token on one side (regex `\b[A-Z][A-Z0-9-]{1,7}s?\b`) whose letters appear as the initials of consecutive words on the other side $\rightarrow$ *abbreviation vs. expansion*; less than half of the query terms present $\rightarrow$ *partial overlap*; otherwise *high overlap but outranked* (the words are there, other documents just score higher: length, competing near-duplicates, or a multi-hop question whose second gold passage shares few words). The document text for the examples comes from `ir_datasets’ docs_store().get(docid)`, because the index does not store raw text.

Dataset	(q,gold) pairs	Failure rate	Jaccard success (mean/med)	Jaccard failure (mean/med)	AUC
---------	----------------	--------------	----------------------------	----------------------------	-----

Table 5: Part 3: BM25 (tuned) failure = gold document not in the top-10. Jaccard overlap of analysed query and gold-document tokens; AUC = probability that a random success pair has higher overlap than a random failure pair.

Dataset	$J=0$	$0 < J < .05$	$.05 \leq J < .10$	$.10 \leq J < .20$	$J \geq .20$
---------	-------	---------------	--------------------	--------------------	--------------

Table 6: Part 3: failure rate (number of pairs) per Jaccard-overlap bin.

Dataset	no overlap	abbrev. vs exp.	partial overlap	high overlap, outranked	total
---------	------------	-----------------	-----------------	-------------------------	-------

Table 7: Part 3: failure cases by (heuristic) category.

7 Part 4a: Rocchio and RM3 (roocchio.py, phase4a.py)

7.1 Concept

Pseudo-relevance feedback assumes the top- N documents of a first search are relevant and uses their vocabulary to re-write the query. Rocchio:

$$w_t = \alpha f(q)[t] + \frac{\beta}{N} \sum_{d \in \text{top-}N} \tilde{f}(d)[t].$$

$f(q)$ is the query's term-frequency vector, $\tilde{f}(d)$ the document's; we divide both by their length so that a long document does not dominate and the query side keeps total weight α . Terms present in more than 10% of the corpus are never added (they carry no information and would pull every query the same way); the k heaviest remaining terms are kept. *Query drift*: when the top- N contain off-topic documents, their words are added and the re-search moves away from the original intent.

RM3 is the probabilistic cousin: it estimates a relevance language model from the feedback documents, keeps the top terms, and interpolates it with the original query ($\lambda = \text{original_query_weight}$). Pyserini implements it in Java behind `set_rm3`.

7.2 Code

```
centroid = Counter()
for vec in vectors: # vectors: term -> tf, one per feedback document
    length = sum(vec.values()) or 1
    for t, tf in vec.items():
        centroid[t] += tf / length
weights = {t: alpha * tf / q_len + beta / n * centroid.get(t, 0.0) for t, tf in q.items()}
expansion = sorted((t for t in centroid if t not in q and not self.too_common(t)), key=
    lambda t: -centroid[t])[:k]
for t in expansion:
    weights[t] = beta / n * centroid[t]

builder = querybuilder.get_boolean_query_builder()
for term, w in weights.items():
    builder.add(querybuilder.get_boost_query(JTermQuery(JTerm("contents", term)), float(w)
        ), JBooleanClauseOccur.should.value)
hits = searcher.search(builder.build(), k=100)
```

Why the query builder instead of pasting words into a string? A string query would be analysed again (double stemming) and could not carry per-term weights; `BoostQuery` multiplies each term's BM25 contribution by w_t . In `phase4a.py` the feedback vectors come from the index (`roocchio.index_vectors(to_docsids)`), the run is evaluated, per-query nDCG@10 is compared with the baseline to list drift candidates, and RM3 is run on the same queries:

```
searcher.set_rm3(fb_terms=k, fb_docs=n_fb, original_query_weight=0.5)
entry["rm3"] = common.evaluate(common.search(searcher, queries), qrels)
searcher.unset_rm3()
```

Dataset	Model	nDCG@10	R@100	MRR@10	MAP
---------	-------	---------	-------	--------	-----

Table 8: Part 4a: own Rocchio ($\alpha=1, \beta=0.75$, df-cutoff 10%) and Pyserini RM3 (original-query weight 0.5) at two (N, k) settings, vs. tuned BM25 on the same queries (FEVER / HotpotQA: fixed random sample of 500 test queries).

8 Part 4b: HyDE (hyde_generate.py, phase4b.py)

8.1 Concept

HyDE (Hypothetical Document Embeddings) asks an LLM to write the document that *would* answer the query, then uses that text for retrieval. Here it replaces the corpus-retrieved feedback documents of 4a: the LLM’s passage is analysed into a term vector and fed to the same Rocchio code. Two ways to combine: *naïve concatenation* (query + passage as one bag of words, so every generated word counts like a query word) versus the *feedback model* (Rocchio keeps the query dominant, filters common words and keeps only k terms). The question the assignment asks is which matters more: where the words come from (LLM vs. corpus) or how they are combined (concatenation vs. Rocchio).

8.2 Code

```
tok = AutoTokenizer.from_pretrained(MODEL, padding_side="left") # left padding for
    batched generation
model = AutoModelForCausalLM.from_pretrained(MODEL, dtype=torch.float32).eval()
prompts = [tok.apply_chat_template([{"role": "user", "content": PROMPT[name].format(q=q.
    text)}]),
    tokenize=False, add_generation_prompt=True) for q in batch
    ]
gen = model.generate(**inputs, max_new_tokens=max_new_tokens, do_sample=True, temperature
    =0.7, top_p=0.9,
    num_return_sequences=n_docs, pad_token_id=tok.pad_token_id)
texts = tok.batch_decode(gen[:, inputs["input_ids"].shape[1]:], skip_special_tokens=True)
    # drop the prompt tokens
```

The script appends one JSON line per query and skips ids already present, so it can be stopped and resumed. The model is Qwen2.5-0.5B-Instruct because it is the largest that runs at a usable speed on this CPU (no GPU, so no vLLM). In `phase4b.py`, `rocchio.text_vectors(hyde[qid])` calls `Analyzer.compute_document_vector` on the generated passages; everything after that is the unchanged 4a code.

Dataset	Variant	nDCG@10	R@100	MRR@10	MAP
---------	---------	---------	-------	--------	-----

Table 9: Part 4b: LLM-generated feedback (Qwen2.5-0.5B-Instruct, 2 hypothetical documents per query) vs. corpus PRF (best 4a setting), same queries as Part 4a.

9 Part 5: SPLADE (phase5.py)

9.1 Concept

SPLADE puts a BERT encoder with its masked-language-model head over the text; for every input position i the head produces a logit o_{ij} for every vocabulary word j . The representation is

$$w_j = \max_i \log(1 + \text{ReLU}(o_{ij})),$$

one weight per vocabulary word, zero for most of them (the FLOPS regulariser during training pushes weights to zero). A word gets weight even if it never appears in the text, because the MLM head predicts related words: that is learned expansion. Documents are encoded once and stored in an *impact index*: a Lucene index whose “term frequency” is the integer weight (Pyserini quantises w_j to integers), built with `-impact -pretokenized`. At query time the query is encoded the same way and the score is the dot product, computed by the usual posting-list traversal. So SPLADE is neural on both sides but still a sparse inverted index at retrieval time.

9.2 Code

```
doc_encoder = SpladeDocumentEncoder(MODEL, device="cpu")
vectors = doc_encoder.encode([d["contents"] for d in batch], max_length=256) # list of {
    word: int weight}
f.write(json.dumps({"id": d["id"], "contents": "", "vector": {t: w for t, w in v.items()
    if w > 0}}) + "\n")
subprocess.run(["python", "-m", "pyserini.index.lucene", "-collection", "
    JsonVectorCollection",
    "-input", coll, "-index", index, "-impact", "-pretokenized"], check=True)
searcher = LuceneImpactSearcher(index, SpladeQueryEncoder(MODEL, device="cpu"))
hits = searcher.batch_search(texts, qids, k=100, threads=8)
```

SciFact (5 183 documents) is encoded locally; FEVER and HotpotQA would take days on a CPU, so their prebuilt `beir-v1.0.0-*.splade-pp-ed` indexes (same checkpoint) are downloaded and only the queries are encoded here. The expansion-term comparison lists SPLADE’s non-zero words that are not literal word-pieces of the query (`encoder.tokenizer.tokenize(text)`), next to Rocchio’s and HyDE’s terms for the same query; all three are passed through the index analyzer (stemming) before counting overlaps, since SPLADE works on unstemmed word-pieces.

Dataset	Model	nDCG@10	R@100	MRR@10	MAP
---------	-------	---------	-------	--------	-----

Table 10: Part 5: SPLADE++ EnsembleDistil (`naver/splade-cocondenser-ensembledistil`) vs. tuned BM25 on the same queries.

Dataset	queries	SPLADE $\cap$ Rocchio	SPLADE $\cap$ HyDE	Rocchio $\cap$ HyDE	all three
---------	---------	-----------------------	--------------------	---------------------	-----------

Table 11: Part 5: average number of shared expansion terms (top-10 per source, compared after Porter stemming).

10 Reading the results

- `results/part1.json`: build time, size; `part1_stats.json`: IndexReader statistics.
- `part2.json`: default/tuned/TF-IDF metrics, the full grid, the chosen k_1, b , BM25 latency.

- **part3.json**: failure rate, Jaccard mean/median for success vs. failure, AUC, failure rate per Jaccard bin, category counts, examples; **part3_jaccard_*.pdf**: histograms.
- **part4a.json**: BM25 vs. Rocchio vs. RM3 per (N, k) , drift candidates, expansion terms of the first 12 queries.
- **hyde_*.jsonl**: generated passages; **part4b.json**: concatenation vs. HyDE+Rocchio vs. corpus PRF.
- **part5.json**: SPLADE metrics, own-index encode/index time and size, per-query expansion rows and average overlaps.

What to expect and why: BM25 tuning moves nDCG@10 by a point or two at most; TF-IDF is clearly below BM25. Most BM25 failures have low Jaccard, and pairs with $J=0$ almost always fail, but many failures still share words (they are *outranked*, not unmatched), so overlap predicts failure strongly but not perfectly. Corpus PRF helps recall more than precision and can hurt SciFact, whose claims are short and specific (drift). HyDE with Rocchio weighting tends to beat naive concatenation: the combination method matters at least as much as the source. SPLADE, trained on MS MARCO, is the strongest system on all three datasets because its expansion is learned rather than guessed from a few documents.

11 Deliverable checklist

Problem statement asks for	Where it is
Corpus size, queries, deviations	report, Datasets paragraph
Index build time and size	Table part1 (index.py)
Default / tuned BM25 / TF-IDF table, k_1, b and justification	Tables part2, part2_grid, Part 2 text
Failure categories with examples, Jaccard comparison, verdict	Tables part3_*, figure, Part 3 verdict
Rocchio and RM3 at $\geq 2 (N, k)$, drift examples	Table part4a, drift list
Concatenation / HyDE-Rocchio / corpus PRF, verdict	Table part4b, Part 4b verdict
SPLADE metrics, 10+ query three-way term table, overlap, disagreements	Tables part5, part5_overlap, part5_terms
LLM usage and chat links	last section of the report

12 Questions to test yourself

1. Which call would fail if the index were built without `-storeDocvectors`?
2. Why is Jaccard computed on analysed tokens rather than raw words?
3. What happens to the Rocchio weights if you remove the division by document length?
4. Why can the same Rocchio class be used for 4a and 4b without any change?
5. Why does SPLADE need `-pretokenized`, and what would the analyzer do to its word-pieces otherwise?
6. Given one query, compute its BM25 score for a two-term document by hand with $k_1=0.9$, $b=0.4$.
7. Change the failure threshold to top-20: which numbers in Part 3 move, and in which direction?